

Week 1

SC concepts

Topic 1: Hard link vs Soft link

Analogy -

- Hard link → two doors leading to **the same room** (even if you remove one door, the room is still accessible).
- Soft link → a **sign pointing to the door** (if the door is gone, the sign leads nowhere).

Feature	Hard Link	Soft Link (Symbolic)
Points to	File's inode (data) inode -> location	File name/path
Inode shared?	Yes	No
Works on directories?	Usually no	Yes
Cross-filesystem?	No	Yes
Survives original file deletion?	Yes	No
Use case	Backup, redundancy	Shortcuts, linking across FS

Hard Link & Disk Space

- Both the original file and the hard link point to the **same inode** (the actual data on disk).
- Creating a hard link **does not consume extra space** for the file data itself—only a tiny bit of space for the new directory entry (the file name).
- If you edit the file via the hard link or the original name, **the same data changes** because they are the same underlying file.

Soft Link & Disk Space

- A soft link **does use a tiny bit of space** (usually a few bytes) to store the **path to the original file**, but it does **not duplicate the file content** either.
- Editing through a soft link affects the original file (if it exists), but deleting the original breaks the soft link.

Topic 2 : Shell built-ins

Shell built-ins are commands that are implemented directly inside the shell (like bash, zsh, or sh) rather than external programs.

- They execute faster than external commands (no new process is spawned).
- They can directly manipulate the shell's environment

Eg - ls, cd -> how to check? Use ``type <command name>``

Note :

- **Built-ins** can do things **external commands cannot**, like directly change the shell's environment or affect the current shell session.
- External commands run in a **separate process** and cannot change the current shell environment directly.

Topic 3: Apropos vs help vs info

Cmd	Scope	Use case	Output style
<code>apropos</code>	All man pages	Search commands by keyword	List of commands & brief descriptions
<code>help</code>	Shell built-ins only	Quick syntax/help for built-in commands	Short explanation & usage
<code>info</code>	Installed software docs	Detailed, structured documentation	Interactive text with sections

Topic 4: File Types

Type	Symbol	Use case
File	-	Files such txt, csv, srt, stc
Directory	d	Folders to organize files
Character device	c	Keyboard, serial ports (byte stream)
Block device	b	Hard drives, USB (block access)
Socket	s	IPC / network communication
Named pipe (FIFO)	p	IPC (streaming data between processes)
-	l	Soft links

Topic 5: Diff between ls -s, stat, du

Command	Shows	Notes
ls -s	Blocks used per file	Quick, less detailed
stat	Detailed file info	Includes exact bytes and metadata
du	Disk usage (file/directory)	Recursive, considers allocated blocks

Week 2 - SSH

Common SSH Errors, Causes, and Fixes

1. Permission denied (publickey)

Error

None

```
Permission denied (publickey).
```

Meaning

The server rejected your SSH key.

Common causes

- Public key not added to the server.
- Wrong username used in SSH command.
- Wrong private key being used.
- Key permissions are incorrect.

Fix

1. Ensure you are using the correct username:

None

```
ssh user@hostname
```

None

2.
Check key permissions:

None

```
chmod 600 ~/.ssh/id_ed25519
```

2. Host key verification failed

Error

None

```
Host key verification failed.
```

or

None

```
WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!
```

Meaning

SSH thinks the server's identity has changed.

Common causes

- The server was reinstalled.
- Server IP changed.
- You are connecting to a different machine.
- Possible security risk (man-in-the-middle).

Fix

Remove the old host key:

None

```
ssh-keygen -R hostname
```

Then reconnect:

None

```
ssh user@hostname
```

3. Connection refused

Error

None

```
ssh: connect to host hostname port 22: Connection refused
```

Meaning

The server is not accepting SSH connections.

Common causes

- SSH service not running.
- Wrong hostname or IP.
- Firewall blocking port 22.

Fix

- Check if the server is running.
- Verify hostname or IP.
- Ensure SSH service is active:

None

```
sudo systemctl status ssh
```

4. Connection timed out

Error

None

```
ssh: connect to host hostname port 22: Connection timed out
```

Meaning

Your system cannot reach the server.

Common causes

- Server is down.
- Network issue.
- Firewall blocking connection.

Fix

- Check internet connection.
- Verify server address.
- Try ping:

None

```
ping hostname
```

5. Could not resolve hostname

Error

None

```
ssh: Could not resolve hostname xyz: Name or service not known
```

Meaning

The hostname is incorrect or unknown.

Common causes

- Typo in hostname.
- DNS not working.
- Server name doesn't exist.

Fix

- Check spelling:

None

```
ssh user@correct-hostname
```

6. No such file or directory (key file)

Error

None

```
No such file or directory: ~/.ssh/id_ed25519
```

Meaning

SSH cannot find your private key.

Common causes

- Key not generated.
- Wrong key path.

Fix

Generate a new key:

None

```
ssh-keygen
```

Or specify the correct key:

None

```
ssh -i ~/.ssh/id_ed25519 user@hostname
```

7. Bad permissions on private key

Error

None

Permissions are too open.

or

None

Unprotected private key file!

Meaning

Your private key file permissions are insecure.

Common causes

- Key is readable by others.

Fix

Set correct permissions:

None

```
chmod 600 ~/.ssh/id_ed25519
```

8. Too many authentication failures

Error

None

Too many authentication failures

Meaning

SSH tried too many keys and got rejected.

Common causes

- SSH agent loaded with many keys.

Fix

Specify the correct key:

None

```
ssh -i ~/.ssh/id_ed25519 user@hostname
```

None

9. WARNING: UNPROTECTED PRIVATE KEY FILE

Error

None

```
WARNING: UNPROTECTED PRIVATE KEY FILE!
```

Meaning

Your private key permissions are too open.

Fix

None

```
chmod 600 ~/.ssh/id_ed25519
```

10. kex_exchange_identification: Connection closed

Error

None

```
kex_exchange_identification: Connection closed by remote host
```

Meaning

Connection was closed before authentication.

Common causes

- Too many connections.
- Firewall rules.
- SSH service issues.

Fix

- Wait and try again.
 - Check server logs.
 - Contact system admin.
-

Quick Troubleshooting Flow

1. **Permission denied**
→ Key issue → copy public key
2. **Host key verification failed**
→ Run:

None

```
ssh-keygen -R hostname
```

3.
Connection refused
→ SSH not running or wrong host
 4. **Could not resolve hostname**
→ Typo or DNS issue
-

Minimal Command Set to Remember

None

```
ssh-keygen          # generate key
ssh user@host       # connect
ssh-keygen -R hostname # fix host key error
chmod 600 ~/.ssh/id_ed25519
```

Week 3 - Networking commands

- **netstat** – Displays active network connections, routing tables, and network interface statistics.
- **ipconfig** – Shows and manages IP configuration of network interfaces (mainly used in Windows).
- **ifconfig** – Displays or configures network interface parameters like IP address and subnet mask (older Linux tool).
- **nslookup** – Queries DNS servers to find the IP address associated with a domain name.
- **ping** – Tests network connectivity to a host by sending ICMP echo requests.
- **tracert** – Shows the path packets take from your system to a destination host.
- **dig** – Advanced DNS lookup tool that provides detailed DNS query results.
- **host** – Performs simple DNS lookups to map domain names to IP addresses.
- **ip** – Modern Linux command used to manage network interfaces, routing, and IP addresses.
- **ss** – Displays socket statistics and network connections (modern replacement for **netstat**).
- **curl** – Transfers data to or from a server using protocols like HTTP, HTTPS, and FTP.
- **wget** – Downloads files from the web using HTTP, HTTPS, or FTP.
- **whois** – Queries domain registration databases to get information about a domain owner and registrar.
- **Wireshark** – A network protocol analyzer used to capture and inspect packets on a network.

Week 4 - Grep Notes

Difference btw whitespace & blank

1. Whitespace

Whitespace means **any character that creates space in text.**

Common whitespace characters:

- Space " "
- Tab `\t`
- Newline `\n`
- Carriage return `\r`
- Vertical tab
- Form feed

Example:

Shell

```
echo "Hello  World"
```

Between `Hello` and `World` there are **whitespace characters**.

In regex tools like `grep`:

Shell

```
grep -E "\s"
```

matches **any whitespace character**.

2. Blank

Blank refers specifically to **horizontal spacing characters only**.

Usually includes:

- Space " "
- Tab `\t`

It **does NOT** include a newline.

Example in `grep`:

```
Shell
grep -E "[[:blank:]]"
```

matches:

```
None
(space)
(tab)
```

but **not newline**.

3. Key Difference

Feature	Whitespac e	Blank
Includes space	✓	✓
Includes tab	✓	✓
Includes newline	✓	✗
Includes vertical/tab/form feed	✓	✗
Scope	broader	narrower

4. Example

File `test.txt`

```
None
hello world
hello world
hello
world
```

Command:

Shell

```
grep -E "[[:blank:]]" test.txt
```

Matches only lines with **space or tab** between words.

But a whitespace-based pattern could also match **newline-separated tokens**.

✓ Simple way to remember

- **Whitespace** = any spacing character
 - **Blank** = only space or tab
-

Week 4 - Grep Practice File

Grep Practice File -

None

```
# System Log - 2025-06-01
INFO 2025-06-01 12:00:00 - System boot initiated
INFO 2025-06-01 12:01:03 - Network initialized
ERROR 2025-06-01 12:02:55 - Failed to connect to 192.168.1.10
WARNING 2025-06-01 12:03:12 - High memory usage detected
```

```
User: Alice Johnson
Email: alice.j@example.com
Phone: +1-555-123-4567
```

```
User: Bob Smith
Email: bob.smith@webmail.org
Phone: +44 20 7946 0958
```

```
User: Charlie Chen
Email: cchen@university.edu
Phone: 123.456.7890
```

```
Random noise: j3lk5j3l5jkl5jkl
```

```
The quick brown fox jumps over the lazy dog.
This is a test line. This is only a test line.
hello Hello HELLO HeLLo
```

```
Match this line only.
Don't match this line.
```

```
192.168.0.1
10.0.0.5
255.255.255.255
```

```
2025-06-01
2024-12-31
1999-01-01
```

```
Line with number: 42
Another number: 1234567890
Special characters: !@#$%^&*()_+=-`~{}[]
```

```
# Indented comment
```

```
End of file.
is this?
This is a line
This is a line

^[[A^[[A
This is a line
This is a line
^[[A^[[B^[[D^[[C^[[A^[[D
```

data.csv

```
None
ID,Name,Email,Phone,Status,IP
1,Alice Johnson,alice.j@example.com,+1-555-123-4567,Active,192.168.1.10
2,Bob Smith,bob.smith@webmail.org,+44 20 7946 0958,Inactive,10.0.0.1
3,Charlie Chen,cchen@university.edu,123.456.7890,Active,172.16.0.5
4,Dana Li,dana.li@startup.co,+1-800-222-3333,Pending,192.168.0.1
5,Eli Zhang,ez@corp.biz,+91-9876543210,Active,8.8.8.8
6,Fay Lin,fay@dev.net,+1-999-999-9999,Inactive,127.0.0.1
7,George T,g.t@aol.com,+61 412 345 678,Pending,192.0.2.0
8,Hannah B,hannahb@site.org,555-321-6789,Active,10.10.10.10
9,Ian K,ian.k@domain.edu,321.654.0987,Active,192.168.100.100
10,Jade Y,jade.y@company.com,+33-1-23-45-67-89,Inactive,198.51.100.1
11,Karen L,karen.l@europe.eu,+49 89 636-48018,Pending,203.0.113.42
12,Luke M,lukem@mail.io,800-555-1234,Active,224.0.0.1
13,Maya N,maya@fashion.net,600-700-8000,Active,150.150.150.150
14,Ned V,nedv@oldmail.com,+81-90-1234-5678,Inactive,172.31.255.255
15,Olive D,olive.d@tech.dev,+1 234 567 8901,Pending,100.100.100.100
16,Paul R,paulr@edu.edu,555-666-7777,Active,192.168.1.1
17,Quinn S,quinns@media.org,+55 11 99999-8888,Active,185.199.110.153
18,Raj T,raj.t@inbox.in,999.888.7777,Inactive,10.0.0.254
19,Samira W,samira.w@global.org,123-321-4567,Pending,172.20.10.5
20,Tom Z,tz@zmail.com,+86 10 1234 5678,Active,192.168.2.2
^
+
I am a student of IIT Madras

0
```

Week 6 - Scripts

Software Tool Principles

- **Do one thing well** → Each tool should focus on a single task and do it efficiently.
- **Process lines of text, not binary** → Tools should handle simple text streams for easy composition.
- **Use regular expressions** → Support pattern matching for flexible text processing.
- **Default to standard I/O** → Read input from stdin and write output to stdout for chaining.
- **Don't be chatty** → Output only necessary information, no extra noise.
- **Generate the same output format accepted as input** → Ensure compatibility for piping between tools.
- **Let someone else do the hard part** → Reuse existing tools instead of reinventing functionality.
- **Detour to build specialized tools** → Create small utilities when needed to simplify complex tasks.

Sourced vs Executed

- **Executed (normal run)** → Runs in a new shell process, so changes (variables, directory, etc.) do not affect the current shell.
- **Sourced (using `source` or `.`)** → Runs in the current shell, so any changes persist in your environment.

Child shell vs sub shell

- **Child shell** → A new independent shell process created to run a command/script; changes don't affect the parent shell.
- **Subshell** → A temporary shell spawned by the current shell (e.g., `( )`); runs in a child environment but is typically used for grouped commands, and changes don't persist outside it.

Different ways of writing mathematical scripts

Shell

```
#!/bin/bash
```

```
a=1
```

```
b=2
```

```
#Using let cmd
```

```
#c = a + b
```

```
let c=$a+$b
```

```
let "c = $a + $b"
```

```
echo "Using let "$c
```

```
#Arithmetic Expansion
```

```
sum=$((a + b))
```

```
k=$((3+4))
```

```
count=0
```

```
(( count++ ))
```

```
echo "Using \${()} "$sum $k $count
```

```
# expr cmd
```

```
c=5
```

```
d=3
```

```
sum=$(expr $c + $d)
```

```
echo "Using expr" $sum
```

```
#bc command
```

```
value=$(echo "scale=2; $d/$c" | bc)
```

```
v=$(( d/c ))
```

```
echo $v
```

```
echo $value
```

```
sum=$(echo "$a + $b" | bc)
```

```
echo "Using bc "$sum
```

If block

Shell

```
#!/bin/bash

read -p "Enter a number: " x
#echo "The number is" $x
if (( x == 0 ));then
    echo "$x is zero"
elif (( x % 2 == 0 )) #we can write == & $x or x can be used!
then
    echo "$x is even"
else
    echo "$x is odd"
fi

#Structure
#a=$(( a + b))
#if -----
#then
#    -----
#    -----
#elif -----
#then
#    -----
#else
#    -----
#fi
```

While loop

Shell

```
#!/bin/bash
```

```
count=5
```

```
while [ $count -gt 0 ] #using $ here is mandatory and spaces  
req after [ and before ]
```

```
do
```

```
    echo "Countdown $count"
```

```
    (( count-- )) #don't put $ here
```

```
    echo "hello"
```

```
    # equivalent to count=$(( count-1 ))
```

```
done
```

```
echo "HI"
```

For loop

Shell

```
for var in $(ls):
```

```
do
```

```
    echo $var
```

```
done
```

```
for (( a=1; a<=10; a++ ))
```

```
do
```

```
    echo $a
```

```
done
```

```
#for i in range(1,10,5) as in python
```

Palindrome script

Shell

```
#!/bin/bash
#this is a shell script

read -p "Enter a word: " str
revr=$(echo "$str" | rev)

if [[ "$str" == "$revr" ]]
then
    echo "Palindrome"
else
    echo "Not a palindrome"
fi
```

Week 7 - Scripts

Week 6 & 7: Scripts

Files used

file

None

total 20

```
-rwxr-xr-x 1 bash bash 101 Nov  6 15:55 c_for.sh
-rwxr-xr-x 1 bash bash 328 Nov  6 15:52 case.sh
-rw-r--r-- 1 bash bash   0 Nov  6 16:13 file
-rw-r--r-- 1 bash bash 260 Nov  6 16:13 file.sh
-rwxr-xr-x 1 bash bash 118 Nov  6 16:10 for2.sh
-rwxr-xr-x 1 bash bash 164 Nov  6 16:09 here.sh
```

file2

None

21

435

234

424

367

54774733STOP

46

STOP

3247072422222222

5555555555555555

5555555555555555

Scripts

Break

Shell

```
#!/bin/bash
```

```
filename=file2
```

```
while IFS= read -r line; do
```

```
    echo "This is a file: " $line
```

```
    if [[ $line == "STOP" ]]; then
```

```
        echo "This is break in use"
```

```
        break
```

```
    fi
```

```
done < $filename
```

Continue

Shell

```
#!/bin/bash

filename=file2

while IFS= read -r line; do

    if [[ $line == "STOP" ]]; then

        echo "This is continue in use"

        continue

    fi

    echo "This is a file: " $line

done < $filename
```

Case statement

Shell

```
#!/bin/bash
echo "What is your favourite cake flavour?"
read flavour
case $flavour in
    [vV]anilla)
        echo "My favourite too"
        ;;
    chocholate)
        echo "Nice"
        echo "That's a good choice"
        ;;
    orange | strawberry)
        echo "weird"
        ;;
    [0-9][0-9])
        (( flavour++ ))

```

```

                echo $flavour
                ;;
        *)
                echo "Incorrect choice"
                ;;
esac

```

Note - case accepts only *?[] (glob) and not regex

Reading from a file

Using while loop for

Shell

```
#!/bin/bash
```

```
filename=file
while read line
do
    echo "While: "$line
done < file

```

```
i=0
```

```
cat file file 2 | while IFS= read -r line #\t -> tab \n -> new line , -> csv
do
    echo "Number $i"
    echo $line
    (( i++ ))
done

```

```
#-----
# Process substitution

```

```
count=0
```

```
echo -e "a\nb\nc" | while read line
do
    ((count++))
    echo $line " : " $count
done
echo "Outside "$count

```

```
#-----

while read line
do
    ((count++))
    echo $line " : " $count
done < <(echo -e "a\nb\nc" )

echo "Outside "$count

#while ---
#do      ....
#        ....
#done < <()
```

Note: It reads line by line

For loop

```
Shell
#!/bin/bash
for line in $(cat file) #can use backticks `` instead of $()
do
    echo $line
done
```

Note: It reads word by word and not line by line

Getopts

Shell

```
#!/bin/bash

while getopts "ab:c:" options;
do

    case $options in

        a)

            echo "Hi";;

        b)

            echo "Argument file - ${OPTARG} ";;

        c)

            echo "File to debug - ${OPTARG}";;

        *)

            echo "invalid";;

    esac

done
```

Heredoc feature

Shell

```
#!/bin/bash

a=2.5
b=3.2
c=4
```

```

#d=$( "echo scale=5; ($a+$b)^$c" | bc )
#heredoc same statement in multiple lines
d=$( bc -l <<- ABC
scale=5
($a+$b)^$c
ABC
)
echo $d

```

Shift

```

Shell
echo "All args before shift: $@"
echo $1
shift
echo "All args after shift: $@"
echo $1
shift
echo "All args after shift: $@"
echo $1

```

Skipping header

```

Shell
#!/bin/bash

flag=true
while read line; do
    if [[ $flag == "true" ]] ; then
        flag=false
        continue
    fi
    echo $line
done < file
#tail -n +2 file | while read ...
#sed 1d file | while ...

```

For loop

Shell

```
#!/bin/bash
```

```
for num in {1..10}
do
    if (( num % 2 == 0 )); then
        #continue
        break
    fi
    echo $num
done
```

Week 9 - Awk Scripts

1. Print specific columns

Question: Print the 1st and 3rd columns from a file.

Solution (script):

```
{ print $1, $3 }
```

2. Filter rows using a condition

Question: Print rows where the second column (age) is greater than 25.

Solution:

```
$2 > 25 { print $1, $2 }
```

3. Count total lines

Question: Count the total number of lines in a file.

Solution:

```
END { print "Total lines:", NR }
```

4. Sum a column

Question: Sum all values in column 2.

Solution:

```
{ sum += $2 }  
END { print "Total:", sum }
```

BEGIN -> initialization, FNR, NR

{ } -> processing,

END { } -> printing something!!!

5. Convert text to uppercase

Question: Convert each line to uppercase.

Solution:

```
{ print toupper($0) }
```

6. Skip comment header lines

Question: Skip lines beginning with # and print the rest.

Solution:

```
$0 !~ /^#/ { print }
```

7. Skip first line (header) and print data

Question: Skip the first line and print all other lines.

Solution:

```
NR == 1 { next }  
{ print }
```

8. Skip header and print Name + Salary

Question: Print name and salary excluding the header.

Solution:

```
NR == 1 { next }  
{ print $1, $4 }
```

9. Skip header and print only Engineers

Question: From the employee dataset, print names of engineers only.

Solution:

```
NR == 1 { next }
$3 == "Engineer" { print $1 }
```

10. Skip header and sum salary

Question: Compute the total salary from column 4 after skipping header.

Solution:

```
NR == 1 { next }
{ total += $4 }
END { print total }
```

11. Skip header and find maximum salary

Question: Print the highest salary.

Solution:

```
NR == 1 { next }
$4 > max { max = $4 }
END { print max }
```

12. Skip header and print rows where age > 30

Question: Print all employees older than 30.

Solution:

```
NR == 1 { next }
$2 > 30 { print }
```

13. Skip header & uppercase names

Question: Convert only the name column to uppercase.

Solution:

```
NR == 1 { next }
{ print toupper($1), $2, $3, $4 }
```

Prime Number AWK Scripts

14. Check if a number is prime

Question: Print “Prime” or “Not Prime” for a given number.

Solution:

```
{
    n = $1
    for(i = 2; i*i <= n; i++)
        if(n % i == 0) exit print "Not Prime"
    print "Prime"
}
```

15. Print only prime numbers from file

Question: Given a file with one number per line, print only primes.

Solution:

```
{
    n = $1
    for(i = 2; i*i <= n; i++)
        if(n % i == 0) next
    print n
}
```

16. Label numbers as “Prime” or “Not Prime”

Question: For each number, print whether it is prime.

Solution:

```
{  
    n = $1  
    prime = 1  
    for(i = 2; i*i <= n; i++)  
        if(n % i == 0) prime = 0  
    print n, (prime ? "Prime" : "Not Prime")  
}
```

Sample Data File for All Exercises

Employee Records

Alice	24	Engineer	55000
Bob	30	Designer	48000
Charlie	29	Manager	62000
David	35	Engineer	70000
Evelyn	28	Analyst	52000
Frank	32	Engineer	64000
Grace	26	Designer	45000
Helen	31	Manager	68000
Ian	27	Analyst	50000
Jenny	34	Engineer	72000

1. Print all rows

None

```
awk -F',' ' '{print}' students.csv
```

2. Print only names

None

```
awk -F',' ' '{print $2}' students.csv
```

3. Print Name + Marks

None

```
awk -F',' ' '{print $2, $4}' students.csv
```

4. Students with marks > 80

None

```
awk -F',' ' '$4 > 80 {print $2, $4}' students.csv
```

5. Students from CSE

None

```
awk -F',' ' '$3 == "CSE" {print $2, $4}' students.csv
```

6. Skip the header

None

```
awk -F',' ' 'NR > 1 {print $2, $4}' students.csv
```

7. CSE students scoring > 80

None

```
awk -F',' ' 'NR > 1 && $3 == "CSE" && $4 > 80 {print $2}'  
students.csv
```


Common AWK Pitfalls for Beginners

Common AWK Pitfalls for Beginners

1. Forgetting that AWK splits columns by whitespace

Beginners often try to do:

```
$0 == "Alice 24 Engineer"
```

AWK automatically splits into \$1, \$2, \$3, ...

Use fields instead:

```
$1=="Alice" && $2==24
```

2. Not quoting regex patterns

Wrong:

```
$0 ~ ^Alice
```

Correct:

```
$0 ~ /^Alice/
```

3. Confusing numeric comparison with string comparison

Wrong (string comparison):

```
$2 == "30"
```

Better:

```
$2 == 30
```

4. Forgetting to escape special regex characters

Wrong:

```
$1 ~ (A+B)
```

Correct:

```
$1 ~ /A\+B/
```

5. Using `print` without comma (forces string concatenation)

Wrong:

```
print $1 $2
```

Output:

```
Alice24
```

Correct:

```
print $1, $2
```

6. Not skipping header lines

Many scripts break because headers contain text.

Fix:

```
NR == 1 { next }
```

7. Treating \$0 like a number

Mistake:

```
if ($0 > 10)
```

`$0` is the whole line (string).
Use specific columns instead.

8. Not resetting variables between runs

When running `awk` on multiple files:

```
sum += $2    # carries over
```

Use:

```
BEGIN { sum = 0 }
```

9. Using uppercase/lowercase incorrectly

Beginners forget AWK is case sensitive:

```
if ($3 == "engineer") # NO
```

Correct:

```
if ($3 == "Engineer")
```

10. Not understanding `next`

`next` jumps to the next line and **skips printing**.

Common mistake:

```
$2 < 20 { next }  
print $1
```

Correct logic but students often don't realize execution stops for that record.

11. Using spaces inside comparison operators

Wrong:

```
if ($2 > 30)    # looks fine but extra spaces may confuse beginners
```

Spacing is allowed, but beginners sometimes break logic like:

```
$2 >30 {print}    # No space before action curly brace
```

Correct:

```
$2 > 30 { print }
```

12. Not using braces for multi-line actions

Wrong:

```
$2 > 30  
print $1
```

This prints always.

Correct:

```
$2 > 30 {  
    print $1
```

```
}
```

13. Forgetting BEGIN and END blocks

Wrong usage:

```
sum=0
{ sum+=$2 }
print sum
```

Correct:

```
BEGIN { sum = 0 }
{ sum += $2 }
END { print sum }
```

14. Misunderstanding field separators for CSV

Beginners use whitespace for CSV files.

Correct for CSV:

```
BEGIN { FS="," }
```

15. Trying to modify original file directly

AWK prints to stdout only.

Beginners think it edits the file.

Correct:

```
awk '{...}' file > newfile
```

Sort & Uniq

The `uniq` command in Unix/Linux **only removes *consecutive* duplicate lines**.

That means if the same line appears multiple times but **not next to each other**, `uniq` won't catch them.

Hence, we typically do:

Shell

```
sort file.txt | uniq
```

Why this works

- `sort` arranges identical lines together.
- `uniq` then removes the repeated consecutive ones.

You should use `uniq` with `sort` because `uniq` only removes *adjacent* duplicates, and sorting ensures that all duplicates become adjacent.

For completeness, if you want to count duplicates:

Shell

```
sort file.txt | uniq -c
```

Or to keep only unique lines (no duplicates at all):

Shell

```
sort file.txt | uniq -u
```

Tmux Config

Shell

```
set -g mouse on
setw -g mode-keys vi

# Set the control character to Ctrl+Spacebar (instead of Ctrl+B)
# set -g prefix C-space
# unbind-key C-b
# bind-key C-space send-prefix

# Set new panes to open in current directory
bind c new-window -c "#{pane_current_path}"
bind '"' split-window -c "#{pane_current_path}"
bind % split-window -h -c "#{pane_current_path}"

bind -n M-H previous-window
bind -n M-L next-window

set-window-option -g mode-keys vi
bind-key -T copy-mode-vi v send-keys -X begin-selection
bind-key -T copy-mode-vi C-v send-keys -X rectangle-toggle
bind-key -T copy-mode-vi y send-keys -X copy-selection-and-cancel

set-option -sa terminal-overrides ",xterm*:Tc"
set -g base-index 1
set -g pane-base-index 1
set-window-option -g pane-base-index 1
set-option -g renumber-windows on
set -g default-terminal "screen-256color"

# List of plugins
set -g @plugin 'tmux-plugins/tpm'
set -g @plugin 'tmux-plugins/tmux-sensible'
set -g @plugin 'christoomey/vim-tmux-navigator'
set -g @plugin 'dreamsofcode-io/catppuccin-tmux'
set -g @plugin 'tmux-plugins/tmux-yank'
set -g @plugin 'tmux-plugins/tmux-resurrect'

# Other examples:
# set -g @plugin 'github_username/plugin_name'
# set -g @plugin 'github_username/plugin_name#branch'
# set -g @plugin 'git@github.com:user/plugin'
# set -g @plugin 'git@bitbucket.com:user/plugin'

set -g @resurrect-strategy-nvim 'session'
```

```
set -g @resurrect-capture-pane-contents 'on'
```

```
# Initialize TMUX plugin manager (keep this line at the very bottom of tmux.conf)  
run '~/.config/tmux/plugins/tpm/tpm'
```

.bashrc file

Shell

```
# ~/.bashrc: executed by bash(1) for non-login shells.
# see /usr/share/doc/bash/examples/startup-files (in the package bash-doc)
# for examples

# If not running interactively, don't do anything
case $- in
    *i*) ;;
    *) return;;
esac

# don't put duplicate lines or lines starting with space in the history.
# See bash(1) for more options
HISTCONTROL=ignoreboth

# append to the history file, don't overwrite it
shopt -s histappend

# for setting history length see HISTSIZE and HISTFILESIZE in bash(1)
HISTSIZE=1000
HISTFILESIZE=2000

# check the window size after each command and, if necessary,
# update the values of LINES and COLUMNS.
shopt -s checkwinsize

# If set, the pattern "**" used in a pathname expansion context will
# match all files and zero or more directories and subdirectories.
#shopt -s globstar

# make less more friendly for non-text input files, see lesspipe(1)
[ -x /usr/bin/lesspipe ] && eval "$(SHELL=/bin/sh lesspipe)"

# set variable identifying the chroot you work in (used in the prompt below)
if [ -z "${debian_chroot:-}" ] && [ -r /etc/debian_chroot ]; then
    debian_chroot=$(cat /etc/debian_chroot)
fi

# set a fancy prompt (non-color, unless we know we "want" color)
case "$TERM" in
    xterm-color|*-256color) color_prompt=yes;;
esac

# uncomment for a colored prompt, if the terminal has the capability; turned
# off by default to not distract the user: the focus in a terminal window
```

```

# should be on the output of commands, not on the prompt
#force_color_prompt=yes

if [ -n "$force_color_prompt" ]; then
    if [ -x /usr/bin/tput ] && tput setaf 1 >&/dev/null; then
        # We have color support; assume it's compliant with Ecma-48
        # (ISO/IEC-6429). (Lack of such support is extremely rare, and such
        # a case would tend to support setf rather than setaf.)
        color_prompt=yes
    else
        color_prompt=
    fi
fi

if [ "$color_prompt" = yes ]; then

PS1='${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\[\033[00m\]:\[\033[01;
34m\]\w\[\033[00m\]\$ '
else
    PS1='${debian_chroot:+($debian_chroot)}\u@\h:\w\$ '
fi
unset color_prompt force_color_prompt

# If this is an xterm set the title to user@host:dir
case "$TERM" in
xterm*|rxvt*)
    PS1="\[\e]0;${debian_chroot:+($debian_chroot)}\u@\h: \w\a\]$PS1"
    ;;
*)
    ;;
esac

# enable color support of ls and also add handy aliases
if [ -x /usr/bin/dircolors ]; then
    test -r ~/.dircolors && eval "$(dircolors -b ~/.dircolors)" || eval
"$(dircolors -b)"
    alias ls='ls --color=auto'
    #alias dir='dir --color=auto'
    #alias vdir='vdir --color=auto'

    alias grep='grep --color=auto'
    alias fgrep='fgrep --color=auto'
    alias egrep='egrep --color=auto'
fi

```

```

# colored GCC warnings and errors
#export
GCC_COLORS='error=01;31:warning=01;35:note=01;36:caret=01;32:locus=01:quote=01'

# some more ls aliases
alias ll='ls -alF'
alias la='ls -A'
alias l='ls -CF'

# Add an "alert" alias for long running commands.  Use like so:
#   sleep 10; alert
alias alert='notify-send --urgency=low -i "$([ $? = 0 ] && echo terminal || echo error)" "$(history|tail -n1|sed -e '\''s/^\s*[0-9]\+\s*//;s/[\;&|]\s*alert$//'\`')"'

alias ss='synchro show'
alias se='synchro eval'
alias si='synchro init'
alias st='synchro status'
alias cc='chmod +x'
ptcc(){
    batcat /opt/cs1102/$(basename $(pwd))/public/*/*
}
ptcs(){
    batcat /opt/se2001/$(basename $(pwd))/public/*/*
}

# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
    if [ -f /usr/share/bash-completion/bash_completion ]; then
        . /usr/share/bash-completion/bash_completion
    elif [ -f /etc/bash_completion ]; then
        . /etc/bash_completion
    fi
fi

```

fi
fi